

TRB-013 — Discovery

Submitter: **Jason Freeman**

Request: *Multiple parsing issues on Lansing Invoices*

What is proposed

Three-part remediation for the Lansing batch failure (root cause: `uq\_invoices\_batch\_page` UNIQUE collision on hard-coded `page\_number=1`, not a vendor-format parsing issue).

****1. Code fix — `netlify/functions/invoice-pipeline-background.ts:304`****

Remove the hard-coded `page\_number: 1` from the single-page (stand-alone PDF) UPDATE payload. Leave `page\_number` NULL for stand-alone uploads; the multi-page splitter path continues to write explicit ordinals (1, 2, 3, ...). Postgres btree UNIQUE treats NULLs as distinct, so multiple stand-alone PDFs in one batch coexist without colliding.

```
```ts
// before
page_number: 1, // by definition page 1 of 1
// after
page_number: null, // only populated by the multi-page splitter
```
```

****2. Schema belt-and-suspenders — `invoices` table****

Rebuild `uq\_invoices\_batch\_page` as a partial unique index so intent is explicit in the schema:

```
```sql
DROP INDEX IF EXISTS uq_invoices_batch_page;
CREATE UNIQUE INDEX uq_invoices_batch_page
ON invoices (batch_id, page_number)
WHERE page_number IS NOT NULL;
```
```

Uniqueness now applies only where it matters (splitter-produced rows). Forensic instrumentation value is preserved.

****3. Recovery — re-parse the stuck Lansing rows (and any prior victims of this bug)****

```
```sql
UPDATE invoices
SET status = 'uploaded',
```

```
parsed_data = parsed_data - 'error'
WHERE status = 'error'
AND parsed_data->>'error' LIKE 'Supabase update failed: duplicate key%uq_invoices_batch_page%';
...
```

The `status='uploaded'` transition re-fires the pipeline trigger; Claude re-extracts cleanly once the code fix is deployed. Frontend upload flow (`src/pages/InvoiceUpload.tsx:159`, shared `batch\_id` per upload action) is unchanged — it was never the bug.

## Desired outcome

Jason can drop any number of single-page Lansing (or any-vendor) PDFs into one upload action and every invoice lands in the database with the data Claude already extracts correctly. The three Elite Home Solutions Lansing invoices (`84133498-00`, `84271095-00`, `84428465-00`) from the 2026-05-19 20:54 UTC batch reach `status='parsed'` with populated `parsed\_data` after the recovery query runs.

## Success criteria

- Re-upload of the original three Lansing PDFs in a single batch produces three `status='parsed'` rows, zero `status='error'` rows, zero `uq\_invoices\_batch\_page` violations in Netlify function logs.
- The recovery `UPDATE` flips the three historical error rows to `parsed`; spot-check confirms `parsed\_data.vendor\_name='Lansing Building Products'` and line items match the PDF.
- Multi-page splitter regression check: upload one known multi-page PDF; resulting rows still carry sequential `page\_number` values (1, 2, 3, ...) and the partial UNIQUE index rejects a deliberate duplicate `(batch\_id, page\_number=2)` insert.
- Jason confirms a fresh end-to-end Lansing batch upload from his account parses without manual intervention.